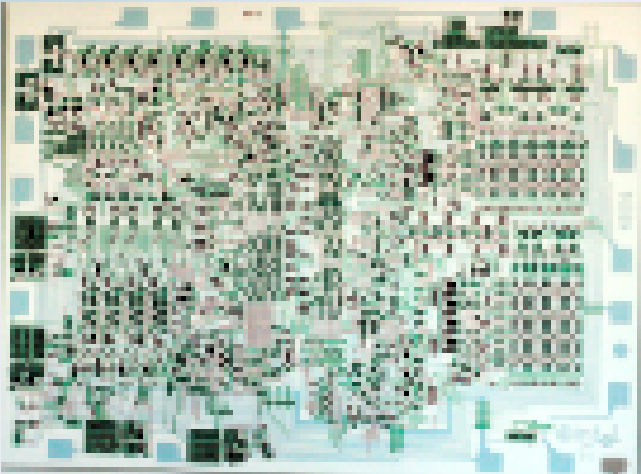




Lecture 12b: Programming in The C Language

**CSCI11: Computer Architecture
and Organization**

Readings

- *Chapters 11-12* [Patt and Patel: Introduction to Computing Systems...](#)
- [The C Language](#) - Kernighan and Ritchie - Second Edition, strongly recommended
- Beej's Guide to C Programming (Tutorial) - Instructor Repo
- Beej's Guide to C Programming (Library Reference) - Instructor Repo

Haptic Presentation

- **Thursday, April 30th from 2 PM to 4 PM at the Sam Karas Room**
- Moshe Helman, Sebastian Rueda-Villamil, and Inam Qasimi to present
- Haptic Vision which is designed to be a natural extension of the body to help the visually impaired navigate the world

Today

- C Pointers
- C Control Structures
- C Calling Functions
- Guide to Writing Problems
- Homework Assignment 12

Pointers in C

A pointer allows us to address memory directly in C.

Declaration:

```
int *p;  /* p is a pointer to an int-type location in memory*/
```

- A pointer in C is **always a pointer to a particular data type**: int*, double*, char*, etc.
- And since it knows its *type*, pointer arithmetic is always accurate

Operators:

- *p -- returns the value pointed to by p
- &z -- returns the address of variable z

Example

```
#include <stdio.h>

void main()
{
    int i, *p_i;
    i = 4;
    p_i = &i;
    *p_i = *p_i + 1;
    printf("%d %d %p %p ", i, *p_i, p_i, &i);
}
```

Indirect Addressing: The Same Idea in Two Languages

Both C and LC-3 support **indirect addressing** — using a location that *holds an address* to reach the actual data.

Concept	LC-3	C
"Pointer" storage	a memory location at a label	a pointer variable (<code>int *p</code>)
Read through pointer	<code>LDI R0, LABEL</code>	<code>x = *p;</code>
Write through pointer	<code>STI R0, LABEL</code>	<code>*p = x;</code>
Get an address	<code>LEA R1, LABEL</code>	<code>p = &x;</code>

In both cases: go to a location → read the address stored there → go to *that* address to get or set the value.

Side by Side: LDI/STI vs. C Pointers

LC-3 — store 42 through a pointer, then load it back:

```
AND  R0, R0, #0
ADD  R0, R0, #42
STI  R0, PTR          ; mem[mem[PTR]] = 42
LDI  R1, PTR          ; R1 = mem[mem[PTR]] → R1 == 42
PTR  .FILL x4000      ; PTR holds address x4000
```

C — same idea with an explicit pointer variable:

```
int  val = 0;
int  *p   = &val;    // p holds the address of val
*p   = 42;           // mem[address in p] = 42 (like STI)
int  r    = *p;       // r = mem[address in p] (like LDI)
```

Both: read the stored address, then access memory at that address

Key Difference: Fixed Label vs. Flexible Variable

- In LC-3, the location that holds the address (PTR) is **fixed at assembly time** — the label cannot change while the program runs.
- In C, the pointer *is a variable* — it can be reassigned at runtime to point to different objects.

```
int a = 1, b = 2;
int *p = &a;
*p = 10;    // a is now 10

p = &b;    // p now points to b – impossible to do with a single LC-3 label
*p = 20;    // b is now 20
```

C pointers give the **same power as LDI/STI**, but the "pointer storage" is a first-class variable you can change, compare, and pass to functions.

Arrays and Pointers: Two Names for the Same Thing

In C, an array name *is* a pointer to its first element — they are interchangeable.

```
int nums[4] = {10, 20, 30, 40};  
int *p = nums;           // p points to nums[0] – no & needed  
  
printf("%d\n", nums[2]); // 30 – array indexing  
printf("%d\n", *(p + 2)); // 30 – pointer arithmetic, same result  
  
printf("%d\n", p[2]);    // 30 – can index a pointer like an array  
printf("%d\n", *(nums+2)); // 30 – can do arithmetic on an array name
```

- $a[i]$ is exactly $*(a + i)$ — the compiler rewrites one as the other
- **Key difference:** an array name is a *constant* pointer — you cannot reassign it (`nums = p` is illegal); a pointer variable can be moved freely

Control Structures

- **Conditional**

- Making decision about which code to execute, based on evaluated expression
- `if`
- `if-else`
- `if else-if else`

- **Iteration**

- Executing code multiple times, ending based on evaluated expression
- `while`
- `for`

If

```
if (condition)  
    action;
```

- Condition is a C expression, which evaluates to TRUE (non-zero) or FALSE (zero).
- Action is a C statement, which may be simple or compound (a block).

Example If Statements

```
// legal, though problematic, don't use
```

```
if (x <= 10)
```

```
    y = x * x + 5;
```

```
// always use {}, thank me, later
```

```
if (x <= 10)
```

```
{
```

```
    y = x * x + 5;
```

```
    z = (2 * y) / 3;
```

```
}
```

Generating Code for If Statement

```
if (x == 2)
{
    y = 5;
}
```

```
LDR  R0, R6, #0    ; load x into R0
ADD  R0, R0, #-2   ; subtract 2
BRnp NOT_TRUE      ; if non-zero, x is not 2
AND  R1, R1, #0    ; store 5 to y
ADD  R1, R1, #5
STR  R1, R6, #1
NOT_TRUE ...       ; next statement
```

If-else

```
if (condition)
    action_if;
else
    action_else;
```

- Else allows choice between two mutually exclusive actions without re-testing condition.

Ultimate form of if-else...

```
if (comparison)
{
    code;
}
else if (comparison)
{
    code;
}
else
{
    code;
}
```

While

```
while (test)  
    loop_body;
```

- Executes loop body as long as test evaluates to TRUE (non-zero)
- Note: Test is evaluated before executing loop body

Generating Code for While

```
x = 0;  
while (x < 10)  
{  
    printf("%d ", x);  
    x = x + 1;  
}
```

Infinite Loops

```
x = 0;  
while (x < 10)  
{  
    printf("%d ", x);  
}
```

- Loop body does not change condition...
- ...so test is never false
- Common programming error that can be difficult to find

For

```
for (init; end-test; step)  
    statement
```

- Executes loop body as long as test evaluates to TRUE (non-zero).
- Initialization and step code included in loop statement.
- Note: Test is evaluated before executing loop body

Example For Loops

```
/* -- what is the output of this loop? -- */  
for (int i = 0; i <= 10; i++)  
{  
    printf("%d ", i);  
}  
  
/* -- what does this one output? -- */  
for (char letter = 'a'; letter <= 'z'; letter++)  
{  
    printf("%c ", letter);  
}
```

Nested Loops

- Loop body can (of course) be another loop

```
/* print a multiplication table */  
for (mp1 = 0; mp1 < 10; mp1++)  
{  
    for (mp2 = 0; mp2 < 10; mp2++)  
    {  
        printf("%d\t", mp1*mp2);  
    }  
    printf("\n");  
}
```

Kernighan and Ritchie Optimization Example

C Programming Language, Kernighan and Ritchie, Second Edition, p. 105-6

```
/* strcpy: copy t to s; array subscript version */
/* must pass as pointers or define i as a argument
void strcpy(char *s, char *t)
{
    int i;
    i = 0;
    while ((s[i] = t[i]) != '\0')
        i++;
}
```

```
/* strcpy: copy t to s; pointer version */
void strcpy(char *s, char *t)
{
    int i;
    i = 0;
    while ((*s = *t) != '\0') {
        s++;
        t++;
    }
}
```

```
/* strcpy: copy t to s; pointer version 2 */  
void strcpy(char *s, char *t)  
{  
    while ((*s++ = *t++) != '\0')  
        ;  
}
```

```
/* strcpy: copy t to s; pointer version 3 */  
void strcpy(char *s, char *t)  
{  
    while (*s++ = *t++)  
        ;  
}
```

For vs. While

- In general:
 - For loop is preferred for counter-based loops
 - Explicit counter variable
 - Easy to see how counter is modified each loop
 - While loop is preferred for sentinel-based loops
 - Test checks for sentinel value.
- Either kind of loop can be expressed as other, so really a matter of style and readability

Break and Continue

`break;`

- used only in *switch* statement or *iteration* statement
- passes control out of the "nearest" (loop or switch) statement containing it to the statement immediately following
- usually used to exit a loop before terminating condition occurs (or to exit switch statement when case is done)

`continue;`

- used only in *iteration* statement
- terminates the execution of the loop body for **this iteration**
- loop expression is evaluated to see whether another iteration should be performed
- if for loop, also executes the step action

Calling Functions in C

- A **function** is a named block of code that performs a specific task
- Functions must be **declared** before they are called
- A **function call** transfers control to the function, then returns

```
return_type function_name(parameter_list); // declaration  
  
result = function_name(arguments);          // call
```

- `return_type` — the type of value the function sends back (`int`, `void`, etc.)
- `arguments` — values passed in to the function

Anatomy of a Function

```
#include <stdio.h>
// Declaration (goes before main)
int square(int n);

// Call (inside main or another function)
int main()
{
    int answer = square(5);    // answer == 25
    printf("%d\n", answer);
    return 0;
}

// Definition
int square(int n)
{
    return n * n;
}
```

Passing Parameters by Value

- Parameters are passed **by value** — the function gets a *copy*
- Changes to a parameter inside the function do **not** affect the caller

```
#include <stdio.h>
void double_it(int x)
{
    x = x * 2;          // only changes the local copy
}
int main()
{
    int a = 4;
    double_it(a);
    printf("%d\n", a);  // still prints 4
    return 0;
}
```

Passing Parameters by Pointer

- Parameters can be passed **by pointer** — the function gets a *reference*
- Changes to the pointer's value inside the function will affect the caller's

```
#include <stdio.h>
void double_it(int *x)
{
    *x = *x * 2;           // value changes due to reference
}
int main()
{
    int a = 4;
    double_it(&a);         // provide address of parameter, not value
    printf("%d\n", a);     // still prints 4
    return 0;
}
```

Returning Values

- Use `return` to send a result back to the caller
- The type of the returned value must match the function's declared return type
- Use `void` when the function has nothing to return

```
int add(int a, int b)
{
    return a + b;    // sends result back
}

int main()
{
    int sum = add(3, 7);    // sum == 10
    printf("Sum: %d\n", sum);
    return 0;
}
```

Putting It All Together

```
#include <stdio.h>

// Declaration
float celsius_to_fahrenheit(float c);

int main()
{
    float temp = 100.0;
    float result = celsius_to_fahrenheit(temp);
    printf("%.1f C = %.1f F\n", temp, result); // 100.0 C = 212.0 F
    return 0;
}

// Definition
float celsius_to_fahrenheit(float c)
{
    return (c * 9.0 / 5.0) + 32.0;
}
```

Quick Guide to Developing C Programs

- Rules for Success
- Template for C Program
- Variables
- Input and Output
- Control

Rules for Success

1. **End each statement with a semicolon (;)**
2. **Include necessary header files** (`#include <stdio.h>` for input/output functions)
3. **Every C program needs a main() function**
4. **Check your braces { },** ensure opening and closing braces are properly paired
5. **Initialize variables before using them**
6. **Use proper data types,** match variable types with their intended use
7. **Add comments to explain your code,** use `//` for ALL comments

Template for a C Program

```
#include <stdio.h>

// Function: main
// Description: counts down from user input to STOP
void main()
{
    // variable declarations

    // prompt user for input

    // perform task

    // print message

}
```

Variables

- **Have to be declared** prior to using
- Keep scope in mind

```
// Integers
int myIntegerVariable

// Characters (not character string)
char myCharacterVariable

// Float
float myFloatVariable
```

Input

```
// simple input
scanf("%d\n", startPoint);

// multiple input, separate entries with a blank
scanf("%d %d", startPoint, counter);
```

Formatting options:

- %d - decimal integer
- %c - ASCII character
- %f - floating-point number

Output

```
// simple print
printf("%d\n", startPoint);

// print expressions, not just variables
printf("%d\n", startPoint - counter);

// print multiple expressions with a single statement
printf("%d %d %d\n", counter, startPoint, counter);
```

Formatting options:

- %d - decimal integer
- %c - ASCII character
- %f - floating-point number

Control Structures

- **Conditional**

- Making decision about which code to execute, based on evaluated expression
- `if`
- `if-else`
- `if-else if-else`

- **Iteration**

- Executing code multiple times, ending based on evaluated expression
- `while`
- `for`

